

Design Specification: Multi-Level Feedback Queue (MLFQ) Scheduler for xv6-RISC-V

Project: Priority-Driven Scheduling for xv6

Implemented By: Mahnoor Mughal (29166), Aabeerah Iqbal (29080)

1. Introduction

To enhance xv6's responsiveness and fairness, this project introduces a Multi-Level Feedback Queue (MLFQ) scheduler as a replacement for the stock round-robin scheduler. The implemented design dynamically adjusts process priorities depending on their CPU usage characteristics and includes a system-wide mechanism to prevent low-priority starvation.

2. Architectural Summary

2.1 Priority Levels and Organization

The scheduler is composed of **four distinct scheduling tiers**, ordered from highest (level 0) to lowest (level 3).

Each tier is internally represented as a **FIFO linked queue**, constructed using the `next_proc` pointer embedded in `struct proc`.

This avoids the overhead of array shifting and keeps queue management efficient.

2.2 Time Slice Distribution

Each priority class receives a different quantum:

Level	Quantum	Notes
0	4 ticks	Highest responsiveness
1	8 ticks	Moderate
2	16 ticks	CPU-friendly
3	32 ticks	Lowest priority

The exponential scaling reduces context-switch overhead for CPU-intensive workloads while still keeping short-running interactive tasks responsive.

3. Process Behavior Rules

3.1 Initialization (New Processes)

- Always inserted into **queue 0**
- Assigned **4-tick** quantum
- `ticks_used = 0`

3.2 Execution and Accounting

During execution:

- The timer interrupt increments `ticks_used`
- Upon yielding the CPU:
 - If the entire quantum was consumed → **demote one level**
 - Otherwise → remain in the same level

`ticks_used` resets every time a process is placed in a different queue.

3.3 I/O Events

- When a sleeping process wakes up, it **re-enters its current queue**
- It is **not** demoted due to I/O blocking
→ supports interactive responsiveness

3.4 Demotion Policy

- Occurs strictly when the full time slice expires
 - No level falls below level 3
 - Quantum is reassigned according to the new level
-

4. Prevention of Starvation

To ensure long-term fairness, the system performs a periodic **global priority reset**:

- Every **200 ticks**, all RUNNABLE and RUNNING processes are reassigned to **queue 0**
- Their quanta are updated
- Running processes (not currently in any queue) have fields updated without re-queueing

This part ensures that low-priority processes eventually regain CPU access.

5. Data Structures and Kernel Extensions

5.1 Additions to `struct proc`

```
int queue_level;           // Current priority tier

int ticks_used;           // Time spent in this tier

struct proc *next_proc;

int quantum;              // Remaining slice for the tier
```

5.2 Queue Representation

```
struct {
```

```
    struct proc *head;

    struct proc *tail;

} mlfq[MLFQ_LEVELS];    // 4 queues
```

5.3 Global Tick Tracker

```
static uint boost_counter;
```

Tracks progress toward the next priority boost.

6. Functional Components

6.1 Queue Operations

- `get_quantum(level)` – Determines time slice for a level
- `mlfq_add_to_queue(p)` – Appends process to target queue
- `mlfq_dequeue(level)` – Retrieves head element
- `mlfq_remove(p)` – Removes from current queue
- `mlfq_next()` – Chooses runnable process from highest available priority

6.2 Scheduling Behavior

- `mlfq_demote(p)` – Moves process one level down
- `mlfq_boost()` – Implements the starvation countermeasure
- `scheduler()` – Central loop, integrates `mlfq_next()`
- `yield()` – Applies demotion rules and enqueues the process

6.3 System Call Support

The system call:

```
getprocinfo(pid, struct procinfo*)
```

Returns:

- PID
- Queue level
- ticks_used
- current quantum
- state
- process name

This is critical for validation and debugging.

7. Execution Flow (Conceptual Algorithms)

7.1 Scheduler Loop

1. Enable interrupts
2. Select next process via MLFQ
3. If found → mark RUNNING and perform context switch
4. If not → wait for next interrupt

7.2 Yield Handling

1. Lock process
2. Examine whether quantum expired
3. Demote if needed
4. Requeue at the appropriate level
5. Perform context switch back to scheduler

7.3 Timer Interrupt

- Increment `ticks_used`

- Increase `boost_counter`
 - Trigger `mlfq_boost()` when limit reached
 - Invoke `yield()`
-

8. Verification Strategy

8.1 Test Categories

1. CPU-Heavy Load

- Expected: moves progressively to level 3
- Boost returns it to level 0 every 200 ticks

2. I/O-Heavy Process

- Expected: remains at level 0
- Avoids unnecessary demotion

3. Mixed Process Set

- CPU-bound → demoted
- I/O-bound → stays at high priority

- Both complete

4. Starvation Safety

- Run several CPU-bound processes
- Confirm all receive CPU due to boosting

8.2 Validation Techniques

- Print process state via `getprocinfo()`
 - Monitor queue transitions
 - Observe tick counters
 - Confirm correctness of demotion, boosting, and re-queuing
-

9. Design Challenges and Resolutions

9.1 Lock Dependencies

- Potential deadlocks during queue manipulation
- Ensured right sequence: acquire process lock before altering queue state

9.2 Processes Running During Boost

- RUNNING processes don't exist in queues
- Solution: update fields without reinserting

9.3 Maintaining I/O Responsiveness

- Avoided demoting processes that block voluntarily
 - Preserved the essence of MLFQ for interactive workloads
-

10. Behavioral Examples

10.1 CPU-Bound Evolution

Time	Level	Ticks	Outcome
0	0	0	Start
4	1	0	Demote
12	2	0	Demote
28	3	0	Demote
60	3	0	Remains

200	0	0	Boosted
-----	---	---	---------

10.2 I/O-Bound Workflow

Time	Level	Ticks	Action
0	0	0	Created
2	0	2	Sleeps
4	0	2	Wakes
6	0	0	Sleeps
8	0	0	Wakes

This illustrates how I/O-bound tasks retain high responsiveness.